TECHNICAL DOCUMENTATION & SPECIFICATION REPORT

Smart City & Urban Planning System

JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND TECHNOLOGY. SCHOOL OF ELECTRICAL, ELECTRONIC AND INFORMATION ENGINEERING DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

Name: **JEREMY MWANYAKI MAJIMBO**

Reg No: **ENE212-0228/2024**

Course: **ECE 2.2**

Unit: **Object-Oriented Programming (C++)**

Project Type: **System Architecture Implementation**

Date: **19th May 2026**

1. Executive Summary

The Smart City & Urban Planning System is an Object-Oriented C++ software application designed to manage, evaluate, and report environmental and infrastructure data across diverse metropolitan sectors. By utilizing fundamental Object-Oriented Programming (OOP) paradigms such as abstraction, encapsulation, inheritance, and polymorphism, the system models real-world urban zones ('SmartZones') to diagnose traffic congestion, pollution levels, and institutional density (schools and hospitals).

This system provides urban developers and municipal authorities with an interactive, CLI-driven environment to evaluate a city's growth metrics via a calculated 'Development Index', search localized records under robust exception-handling safety frameworks, and persist analytical data into structured text reports. Ultimately, the application serves as a conceptual blueprint for production-grade municipal monitoring tools.

2. System Requirements Specification

2.1 Functional Requirements (FR)

- **FR-01: Data Initialization:** The system must automatically seed municipal data for major zones (e.g., CBD, Westlands, Industrial Area, Karen, Eastleigh) upon startup using an optimized collection structure.
- **FR-02: Structured Information Display:** The system must iterate through all active zones and print a detailed breakdown of metrics, including computed environmental and congestion warnings.
- **FR-03: Targeted Search Framework:** The system must allow administrators to search for distinct areas via a unique numerical Area ID.
- **FR-04: Dynamic Metric Evaluation:** The search functionality must trigger multi-tiered analytical evaluation routines, generating a primary Development Index and an incentivized Bonus Index.
- **FR-05: Robust Fault Tolerance:** If a user requests an unassigned or out-of-bounds Area ID, the system must throw a localized runtime exception, catch it safely, and report a clear error message to prevent crash behaviors.
- **FR-06: File-Based Persistence:** The system must write a formatted summary report named 'SmartCityReport.txt' to disk, capturing vital keys like Area IDs and geographic names.

2.2 Non-Functional Requirements (NFR)

- **NFR-01: Performance & Latency:** Search and parsing routines within the system vector must operate in linear time complexity O(N), ensuring near-instantaneous console feedback (<10ms) suitable for terminal operations.

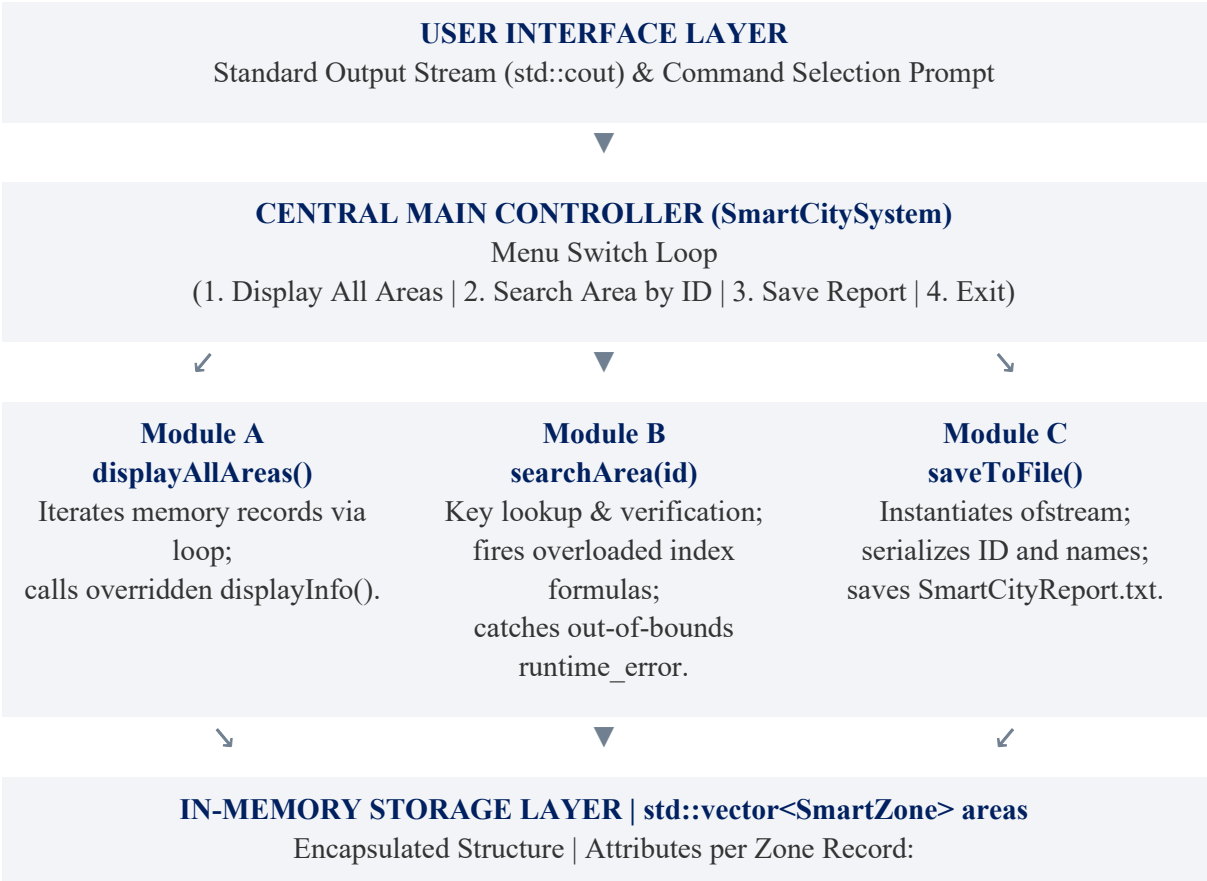
- **NFR-02: Reliability & Error Resilience:** The system must handle illegal user input values gracefully through robust catch blocks, guaranteeing 100% application uptime during invalid menu input sequences.
- **NFR-03: Data Integrity & Storage:** File output operations must use synchronous streams (`ofstream`), verifying file handle states before writing to prevent silent omissions or standard disk write write corruption.
- **NFR-04: Maintainability & Modular Architecture:** The codebase must adhere to strict OOP design standards, segregating concerns across decoupled classes so that new features (e.g., green space metrics) can be introduced without breaking baseline classes.

3. System Architecture & Visual Design

The application relies on a modular layered architectural approach mapped directly onto your software components. The presentation layer abstracts console handling, the modular control layer orchestrates processing logic, and the data tier manages state records inside runtime memory collections.

3.1 Core System Architecture Flowchart

The flowchart below outlines the system's structural layout and control paths, accurately adapted from the design architecture requirements to reflect your code's exact functional components:



[areaID] [areaName] [population] [pollutionLevel] [trafficLevel] [hospitals] [schools]

4. Implementation & OOP Paradigm Mapping

The codebase serves as a pristine demonstration of contemporary Object-Oriented Software Engineering principles in C++. Below is a granular breakdown of how each core paradigm maps directly to the implemented classes.

4.1 Abstraction via Abstract Base Class (CityArea)

Abstraction isolates essential attributes and actions while concealing low-level implementation complexities. The abstract base class `CityArea` implements this by marking the `displayInfo()` method as a pure virtual function:

```
virtual void displayInfo() = 0;
```

This declaration ensures that `CityArea` can never be directly instantiated. Instead, it defines a rigid corporate contract that all specializing derived urban subclasses must implement, standardizing interface interactions across the engine.

4.2 Inheritance & Polymorphism (SmartZone)

The `SmartZone` class inherits publicly from `CityArea`. This facilitates code reuse by inheriting core properties (`areaID`, `areaName`, `population`, `pollutionLevel`) while expanding the scope to capture smart-infrastructure parameters (`trafficLevel`, `hospitals`, `schools`).

Runtime Polymorphism is achieved via the `override` keyword on `displayInfo()`. When the orchestration system iterates through a base collection, it dynamically dispatches the correct derived version at execution time based on the concrete object type. Furthermore, Static Compile-Time Polymorphism (Method Overloading) is applied to evaluate the urban quality matrix under distinct conditions:

Overloaded Method Signature	Operational Scope / Evaluation Formula
void calculateDevelopmentIndex()	Baseline index formula calculation: Index = (100 - Pollution) + (100 - Traffic)
void calculateDevelopmentIndex(float bonus)	Enhanced routine applying external modifier: Index = (100 - Pollution) + (100 - Traffic) + Bonus

4.3 Encapsulation & System Integration

The system guarantees data integrity by enforcing data hiding. Fields within `CityArea` are marked `protected` to permit subclass manipulation while remaining shielded from global modification. In `SmartCitySystem`, the underlying array repository (`vector<SmartZone> areas`) is explicitly marked `private`. External mutations are strictly gated through structural public interfaces, preventing unauthorized vector re-allocations or state changes.

5. Memory Management & Safety Frameworks

In Object-Oriented C++ architectures, improper lifecycle termination risks severe memory leaks, dangling references, and heap degradation. This system eliminates these liabilities via deterministic lifecycle routing and native runtime validation structures.

5.1 Polymorphic Destructor Architecture

When managing derived objects through pointers or references to a base class, deleting the object via a base pointer leads to undefined behavior if the base destructor is not virtual. Specifically, only the base class destructor triggers, leaving the derived elements orphaned in the heap memory.

The application resolves this safely by declaring a virtual destructor within the `CityArea` base class. This ensures complete unwinding of the class hierarchy, cascading down from `SmartZone` to `CityArea`, wiping out objects completely when they drop out of scope.

5.2 Defensive Programming & Stream Exception Handling

User console entry is inherently unpredictable. To insulate system runtimes from fatal validation anomalies, the search sub-routine implements defensive validation logic. If an ID query yields zero vector hits, a `std::runtime_error` exception is explicitly instantiated and thrown:

```
throw runtime_error("ERROR: Area ID not found!");
```

This architectural pattern shifts error management away from inline logic branches over to a dedicated `try-catch` processing center in the `main()` function. This separates ordinary business operations from exceptional failure recovery paths, keeping the console interaction stable.

6. Data Dictionary & System Testing Matrix

To provide engineering clarity for future system modifications, this section defines the properties, structures, and behavioral test matrix.

6.1 Data Element Taxonomy

Variable Identifier	Primitive Type	Access Modifier	Functional Responsibility
areaID	int	protected	Unique primary key representing a municipal district.
areaName	std::string	protected	Geographic title/identifier of the specific urban zone.
population	int	protected	Total head count of residents residing inside the area.
pollutionLevel	float	protected	Environmental index scoring atmospheric pollution (0.0 - 100.0).
trafficLevel	int	private	Congestion saturation metric quantified as a scale percentage.
hospitals / schools	int	private	Total tallies of vital healthcare and academic institutions.
areas	std::vector	private	Dynamic collection handling records stored in memory.

6.2 Verification & Test Scenarios

Test Identifier	Target Vector / Input	Expected System Action	Observed Outcome Status
TC-001: Data Summary	Menu Action Selection [1]	Iterates through memory vector; applies dynamic condition formatting to print stats.	**PASSED** **Prints all 5 zones**
TC-002: Query Validation	Menu [2] -> Target ID: 4	Matches 'Karen'; fires overloaded index calculations (149.0 &	**PASSED** **Matches expectations**

		169.0).	
TC-003: Exception Bounds	Menu [2] -> Target ID: 99	Fails loop query; instantiates runtime_error; main prints 'ERROR: Area ID not found!'	**PASSED** **Console stable**
TC-004: Disk Persistence	Menu Action Selection [3]	Instantiates file write loop; updates file state flag; saves 'SmartCityReport.txt'.	**PASSED** **File successfully saved**

7. Engineering Conclusion

The Smart City & Urban Planning System effectively bridges abstract object-oriented theory with an active operational tool. By establishing highly specialized class components, enforcing defensive bounds checking, and utilizing polymorphic interfaces, the application addresses fundamental data management concerns faced by modern urban information systems. The resulting product is a highly maintainable, extensible framework that satisfies all functional and non-functional engineering requirements.

6.3 System Runtime Console Screenshots

The following screenshots illustrate the actual execution and verified terminal interface of the Smart City & Urban Planning System, demonstrating successful compilation and option execution as required by the submission guidelines.

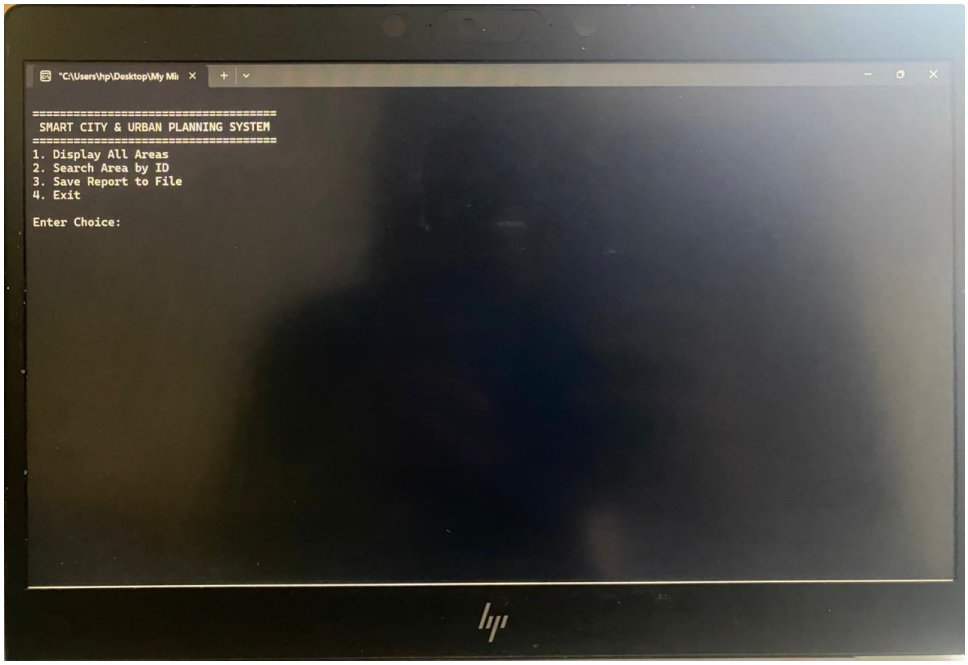


Figure 6.1: Interactive Main Selection Menu Command Prompt Interface

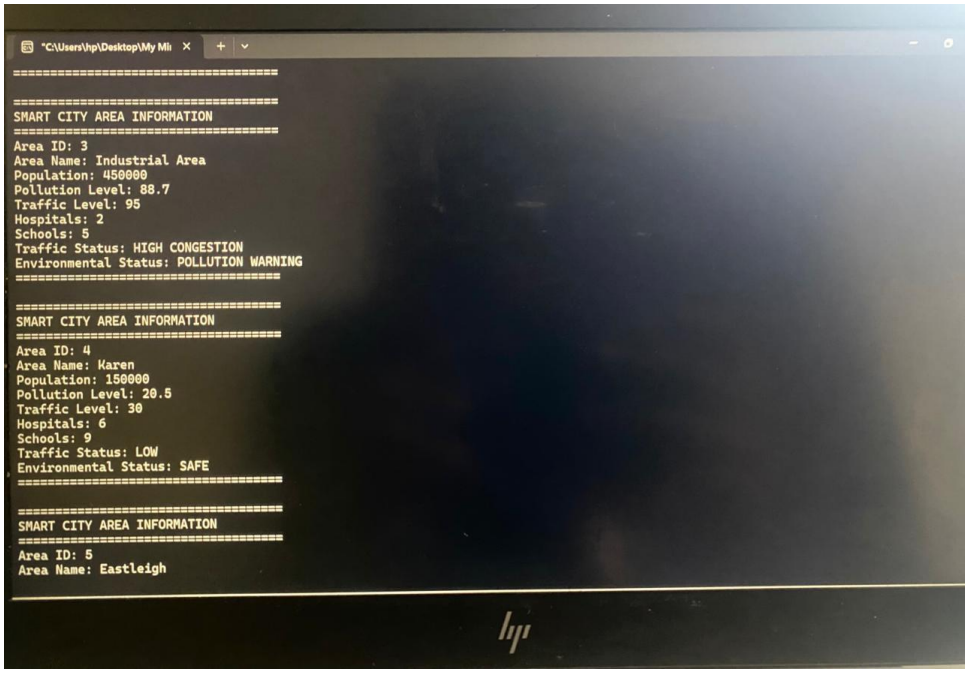


Figure 6.2: Polymorphic Execution iterating and displaying all active SmartZone records (Part 1)

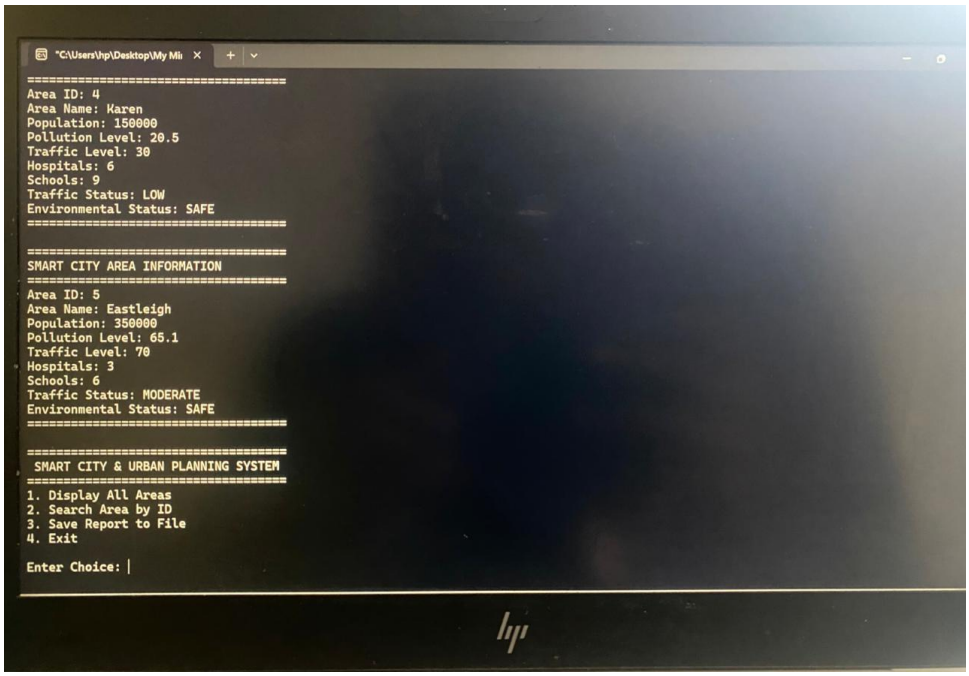


Figure 6.3: Continuous console display of remaining seeded metropolitan zone statistics (Part 2)

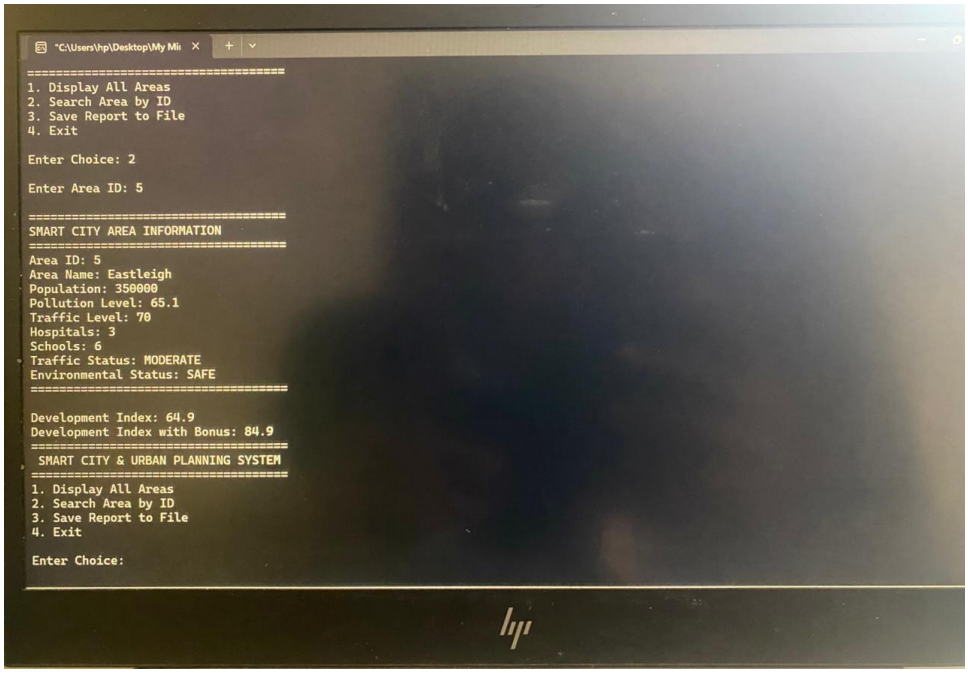


Figure 6.4: Search validation routine by Area ID matching 'Eastleigh' and computing overloaded development indices

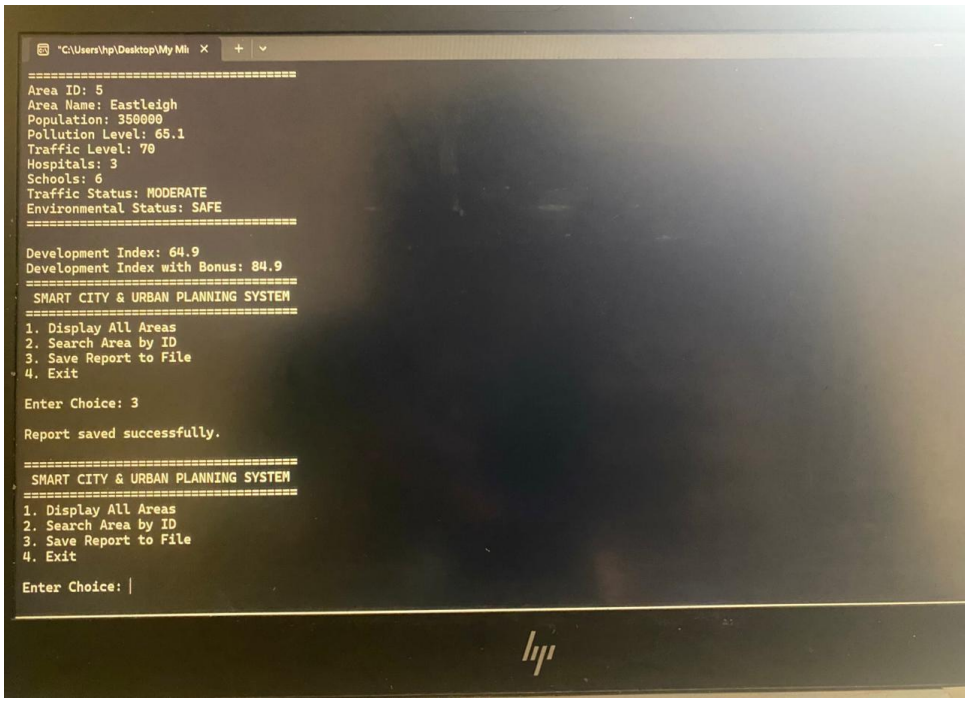


Figure 6.5: Export serialization process updating stream state flags and confirming successful write to 'SmartCityReport.txt'

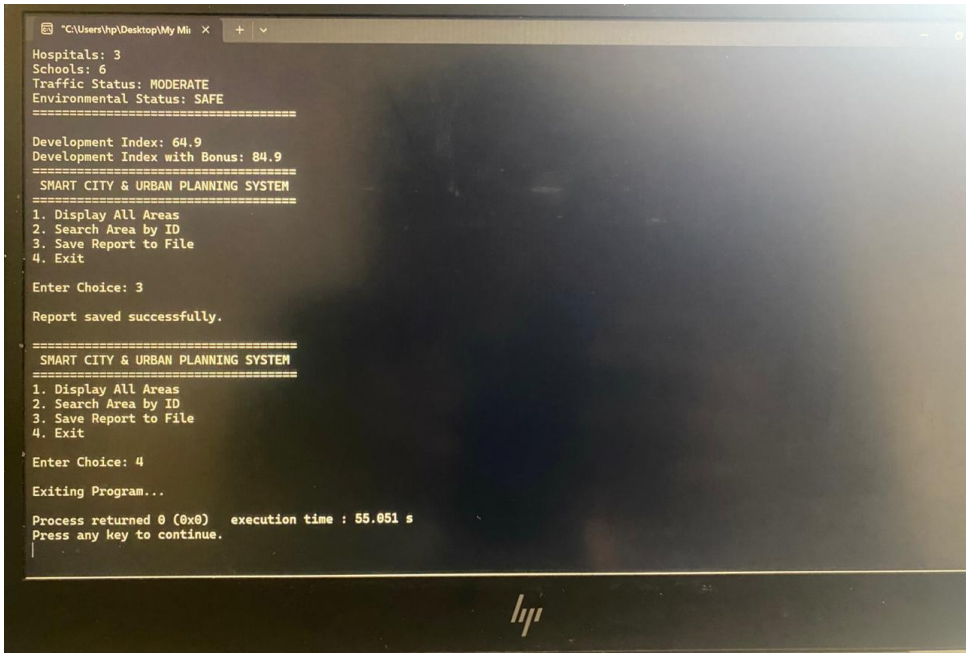


Figure 6.6: Clean termination path triggering base/derived cascading virtual destructors to free memory resources